

A ACCURACY ANALYSIS

THEOREM A.1. Suppose the true range count is RC , the estimated value is $\hat{RC}$, and the false positive rate of the Bloom filters is f_p . Define $\beta = (f_p)^2$, $t = \frac{\beta(N-RC)}{S}$, and $w = (1 - \frac{1}{S})^{RC}$, where N is the number of records in the union of $\{D_i\}_{i=1,\dots,I}$, and S is the counter length of the LC sketch. Then the estimator used in PPRC satisfies

$$\text{Bias}(\frac{\hat{RC}}{RC}) = \mathbb{E}(\frac{\hat{RC}}{RC} - 1) = \frac{2\beta N + w^{-1}e^{-t} - 1}{2RC}, \quad (3)$$

$$\text{StdError}(\frac{\hat{RC}}{RC}) = \text{Std}(\frac{\hat{RC} - RC}{RC}) = \frac{\sqrt{S(\frac{e^t}{w} - 1)}}{RC}. \quad (4)$$

PROOF. After the PRP Execution, there are $RC + r$ objects that are determined within the query range R . Here, r represents the number of false positives—records that are mistakenly determined to be within R due to the probabilistic nature of the Bloom filter tests. From the definition of the false positive rate f_p , we have $\mathbb{E}(r) = (f_p)^2 N = \beta N$ because we use two independent Bloom filters to conduct the membership test. Furthermore, r follows a binomial distribution: $r \sim \binom{N-RC}{r} (\beta)^r (1-\beta)^{N-RC-r}$.

Then, during OLC, PPRC uses the LC sketch to estimate $RC + r$. Let A_s be the event that counter s is empty in the aggregated sketch LC, and let 1_{A_s} be the corresponding indicator random variable. Since the hash function outputs the uniform random number, $P(A_s) = (1 - \frac{1}{S})^{RC+r}$, and $P(A_{s_1} \cap A_{s_2}) = (1 - \frac{2}{S})^{RC+r}$, $s_1 \neq s_2$. Because S' is the number of zero counters in the LC sketch, we have: $S' = \sum_{s=1}^S 1_{A_s}$. Since PRP and OLC are independent, we have:

$$\begin{aligned} \mathbb{E}(S') &= \sum_{r=0}^{N-RC} \sum_{s=1}^S P(A_s) \binom{N-RC}{r} (\beta)^r (1-\beta)^{N-RC-r} \\ &= \sum_{r=0}^{N-RC} S(1 - \frac{1}{S})^{RC+r} \binom{N-RC}{r} (\beta)^r (1-\beta)^{N-RC-r} \\ &= Sw \sum_{r=0}^{N-RC} \binom{N-RC}{r} (\beta - \frac{\beta}{S})^r (1-\beta)^{N-RC-r}. \end{aligned}$$

According to the binomial theorem, we have $\sum_{r=0}^{N-RC} \binom{N-RC}{r} (\beta - \frac{\beta}{S})^r (1-\beta)^{N-RC-r} = (1 - \beta + \beta - \frac{\beta}{S})^{N-RC} = (1 - \frac{\beta}{S})^{N-RC}$, and thus $\mathbb{E}(S') = Sw(1 - \frac{\beta}{S})^{N-RC}$. Since $\frac{\beta}{S} \ll 1$, applying the exponential approximation $(1-x)^n \approx e^{-nx}$ yields $\mathbb{E}(S') \approx S w e^{-t}$, where $t = \frac{\beta(N-RC)}{S}$. Similarly, we have:

$$\begin{aligned} \mathbb{E}((S')^2) &= \mathbb{E}((\sum_{s=1}^S 1_{A_s})^2) \\ &= \mathbb{E}(\sum_{s=1}^S (1_{A_s})^2) + 2\mathbb{E}(\sum_{s_1=1}^S \sum_{s_2=1}^{s_1-1} 1_{A_{s_1}} 1_{A_{s_2}}) \\ &= \mathbb{E}(\sum_{s=1}^S (1_{A_s})^2) + 2\mathbb{E}(\sum_{s_1=1}^S \sum_{s_2=1}^{s_1-1} 1_{A_{s_1} \cap A_{s_2}}) \\ &= Sw(1 - \frac{\beta}{S})^{N-RC} + (S^2 - S)w^2(1 - \frac{2\beta}{S})^{N-RC} \\ &\approx S w e^{-t} + (S^2 - S)w^2 e^{-2t}. \end{aligned}$$

Thus, $\text{Var}(S') = \mathbb{E}((S')^2) - (\mathbb{E}(S'))^2 = S w e^{-t} - S w^2 e^{-2t}$. Based on $\mathbb{E}(S')$ and $\text{Var}(S')$, we let $V = \frac{S'}{S}$, and we can obtain $\mathbb{E}(V) = w e^{-t}$ and $\text{Var}(V) = \frac{1}{S} w e^{-t} - \frac{1}{S} w^2 e^{-2t}$. Denote $-\ln V$ as $f(V)$ and ρ as $\mathbb{E}(V)$. Using the Taylor series to expand $\hat{RC} = S f(V)$, we have:

$$\begin{aligned} \hat{RC} &= S f(V) \\ &= S \left(f(\rho) + (V - \rho) f'(\rho) + \frac{1}{2} (V - \rho)^2 f''(\rho) + \dots \right) \quad (5) \\ &\approx S \left(f(\rho) - \frac{V - \rho}{\rho} + \frac{(V - \rho)^2}{2\rho^2} \right) \end{aligned}$$

We truncate Eq. (5) after the third term since the expected value of the second term is 0. Then, $\mathbb{E}(\hat{RC}) = S(f(\rho) + \frac{1}{2\rho^2} \mathbb{E}((V - \rho)^2))$. Since $f(\rho) = -\ln \mathbb{E}(V) = t + RC \ln \frac{S}{S-1} = \frac{\beta(N-RC)}{S} - RC \ln(1 - \frac{1}{S})$, using $\ln(1-x) \approx -x$ for small x gives $f(\rho) \approx \frac{\beta(N-RC)}{S} + \frac{RC}{S}$. With $\rho = \mathbb{E}(V) = w e^{-t}$ and $\mathbb{E}((V - \rho)^2) = \text{Var}(V) = \frac{1}{S} w e^{-t} - \frac{1}{S} w^2 e^{-2t}$, it follows that $S f(\rho) \approx RC + \beta(N - RC)$ and $S \frac{1}{2\rho^2} \mathbb{E}((V - \rho)^2) = \frac{w e^{-t} - w^2 e^{-2t}}{2 w^2 e^{-2t}} = \frac{w^{-1} e^t - 1}{2}$.

Then, $\mathbb{E}(\hat{RC}) = RC + \beta(N - RC) + \frac{w^{-1} e^t - 1}{2} \approx RC + \beta N + \frac{w^{-1} e^t - 1}{2}$ since $RC \ll N$. Therefore, $\text{Bias}(\frac{\hat{RC}}{RC}) = \mathbb{E}(\frac{\hat{RC}}{RC} - 1) = \frac{2\beta N + w^{-1} e^t - 1}{2RC}$.

We truncate Eq. (5) after the second term: $\hat{RC} = S(f(\rho) + (V - \rho) f'(\rho)) = S(f(\rho) - \frac{V - \rho}{\rho})$. Therefore, $\text{Var}(\frac{\hat{RC}}{RC} - 1) = \text{Var}(\frac{\hat{RC}}{RC}) = \frac{S^2}{RC^2 \rho^2} \text{Var}(V - \rho) = \frac{S^2}{RC^2 \rho^2} \text{Var}(V) = \frac{S(\frac{e^t}{w} - 1)}{RC^2}$. Taking the square root yields the standard error: $\text{StdError}(\frac{\hat{RC}}{RC}) = \frac{\sqrt{S(\frac{e^t}{w} - 1)}}{RC}$. $\square$

B EXPERIMENTS FOR GOWALLA

In this section, we present the experimental results for the Gowalla dataset [9]. The false positive rate of all Bloom filters is set to 0.0001. The results consistently show that PPRC is several times more accurate and dozens of more efficient than the other protocols.

B.1 Accuracy Comparison

B.1.1 The Effect of I . Fig. 12(a)-(b) report MAE and MRE as the number of DHs varies over $I \in \{5, 10, 15, 20\}$. Here we set the $P = 0.1$ and $S = 2K$.

As I increases, MAE increases while MRE decreases for PPRC, FM-Count, and LL-count; for RCC, both the MAE and MRE increase. PPRC is several times more accurate than RCC, FM-Count, and LL-Count. For example, in Fig. 12(a), the MAE of PPRC is 23.65, 38.93, 46.39, and 51.31 for $I = 5, 10, 15, 20$, respectively. On average, PPRC is 19.07, 1.93, and 2.73 times more accurate than RCC, FM-Count, and LL-Count. Fig. 12(b) corroborates these findings.

B.1.2 The Effect of P . Fig. 12(c)-(d) show MAE and MRE as P varies over $\{0.05, 0.1, 0.15, 0.2\}$, where P denotes the proportion of each DH's private dataset relative to the whole Gowalla dataset. We fix $I = 10$ and $S = 2K$.

With increasing P , MAE increases and MRE decreases for PPRC, FM-Count, and LL-Count, whereas both MAE and MRE increase for RCC. Our PPRC remains several times more accurate than RCC, FM-Count, and LL-Count. Fig. 12(c) presents the MAE results. When P is 0.05, 0.1, 0.15, and 0.2, the MAE of PPRC is 21.95, 38.93, 43.91, and 56.85, respectively. On average, PPRC is 18.67, 1.92, and 2.76

times more accurate than RCC, FM-Count, and LL-Count. Fig. 12(d) shows consistent results.

B.1.3 The Effect of S . Fig. 12(e)-(f) evaluate the effect of the counter length of the count estimation sketches $S \in \{1K, 2K, 3K, 4K\}$. The results for RCC remain constant as they do not use the count estimation sketch. Here we set the $P = 0.1$ and $I = 10$.

MAE and MRE of PPRC, FM-Count, and LL-Count decrease with larger S . Similarly, PPRC is several times more accurate than RCC, FM-Count, and LL-Count. As shown in Fig. 12(e), the MAE of PPRC decreases from 46.51 to 31.59 as S increases. Furthermore, on average, our PPRC is 14.47, 1.85, and 2.66 times more accurate than RCC, FM-Count, and LL-Count, respectively. Similar trends are observed in Fig. 12(f).

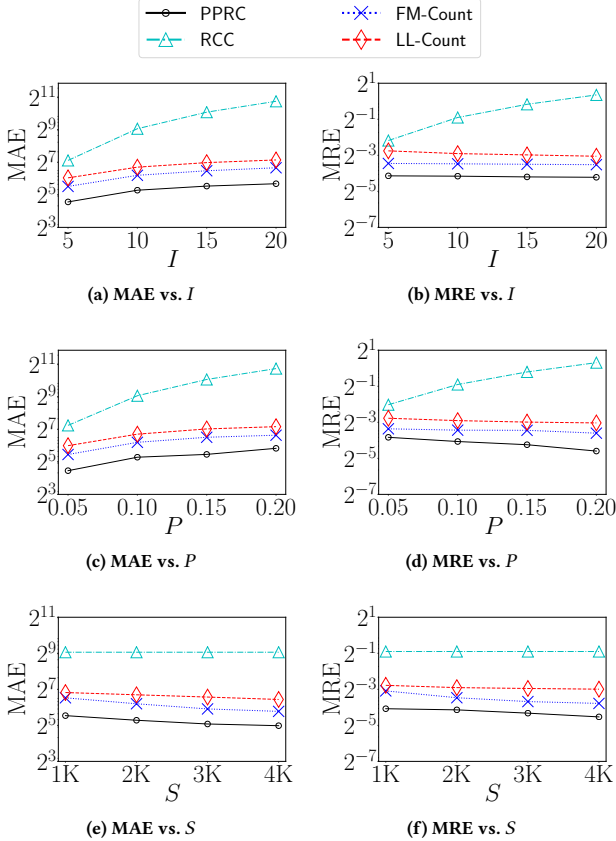


Figure 12: Accuracy comparison with various I , P , and S .

B.2 Efficiency Comparison

B.2.1 Time Cost. Figs. 13(a) and 13(c) show the Time cost with respect to the number of DHs $I \in \{5, 10, 15, 20\}$ and the data proportion $P \in \{0.05, 0.1, 0.15, 0.2\}$. We set $I = 10$, $P = 0.1$, and $S = 2K$ as default.

As I and P increase, the Time cost of all protocols grows. Furthermore, PPRC is dozens of times faster than the other protocols. For example, in Fig. 13(a), the Time cost of PPRC is 1.957s, 1.962s, 1.966s, and 1.972s for $I = 5, 10, 15, 20$, respectively. On average, PPRC is 10.11, 22.00, 23.07, 22.83, and 42.67 times faster than RCC,

LC-Count, FM-Count, LL-Count, and TVA, respectively. Consistent results are shown in Fig. 13(c).

B.2.2 Communication Cost. Figs. 13(b) and 13(d) present the Communication cost concerning $I \in \{5, 10, 15, 20\}$ and $P \in \{0.05, 0.1, 0.15, 0.2\}$. We also set $I = 10$, $P = 0.1$, and $S = 2K$ as default.

As I increases, the communication cost of all protocols increases. As P increases, the communication of TVA increases while the communication cost of the other protocols remains constant. Furthermore, the communication cost of PPRC is several times lower than LC-Count, FM-Count, LL-Count, and TVA. Fig. 13(b) presents the Communication cost with various I . When I is 5, 10, 15, 20, the communication cost of PPRC is 34.52MB, 63.28MB, 92.05MB, and 120.81MB, respectively. On average, PPRC reduces the communication cost by 4.75, 4.79, 4.75, and 19.31 times compared with LC-Count, FM-Count, LL-Count, and TVA. Although RCC achieves lower communication costs, it does not provide the same level of accuracy and computational efficiency as PPRC. Consistent results are shown in Fig. 13(d).

B.2.3 The Effect of S . Fig. 13(e)-(f) evaluate the impact of the counter length of count estimation sketches S on Time and Communication costs. Here, we consider $S \in \{1K, 2K, 3K, 4K\}$, with $I = 10$ and $P = 0.1$. Since RCC and TVA do not use count estimation sketches, their costs remain constant regardless of S .

The Time cost of PPRC, LC-Count, FM-Count, and LL-Count increases with S . Furthermore, PPRC is dozens of times faster than RCC, LC-Count, FM-Count, LL-Count, and TVA. In Fig. 13(e), Time of PPRC is 1.923s, 1.965s, 2.000s, and 2.013s for $S \in \{1K, 2K, 3K, 4K\}$, respectively. On average, PPRC is 10.05, 27.43, 28.15, 27.66, and 30.01 times faster than RCC, LC-Count, FM-Count, LL-Count, and TVA. Fig. 13(f) shows similar patterns for the Communication cost.

C ADDITIONAL RESULTS

Bloom filter size and encryption overhead. Our original SHE construction does not support SIMD-style ciphertext packing, since it operates on modular integer ciphertexts rather than polynomial-ring ciphertexts as in BFV/BGV. Therefore, each Bloom filter bit is encrypted individually. To quantify the resulting overhead, we report the number of inserted points, the plaintext Bloom filter size, the encrypted Bloom filter size, the Bloom filter encryption time, and the total communication cost and communication time of the query under different query ranges. As shown in Table 4, the Bloom filter size grows linearly with the query range, since larger query ranges contain more inserted points. Specifically, when the query range increases from 1 km to 5 km, the number of inserted points increases from 50 to 250, the plaintext Bloom filter size increases from 140 B to 700 B, and the encrypted Bloom filter size increases from 1.09 MB to 5.47 MB. The total communication cost is approximately twice the encrypted Bloom filter size, since the query message includes two encrypted Bloom filters. Accordingly, the Bloom-filter encryption time and communication time also increase proportionally, from 33.4 ms and 8.93 ms to 167 ms and 49.5 ms, respectively.

Encrypted OLC sketch size. The final encrypted result returned to the QU is the aggregated encrypted OLC sketch $E(LC)$, which consists of S encrypted counters. Following the SHE parameter

Table 4: Bloom filter size and encryption overhead under different query ranges.

Range (km)	#Points	Plaintext (B)	Ciphertext (MB)	Enc. Time (ms)	Total Comm. (MB)	Comm. Time (ms)
1	50	140	1.09	33.4	2.19	8.93
2	100	280	2.19	66.5	4.38	17.1
3	150	420	3.28	99.1	6.57	26.5
4	200	560	4.38	131	8.76	35.3
5	250	700	5.47	167	10.95	49.5

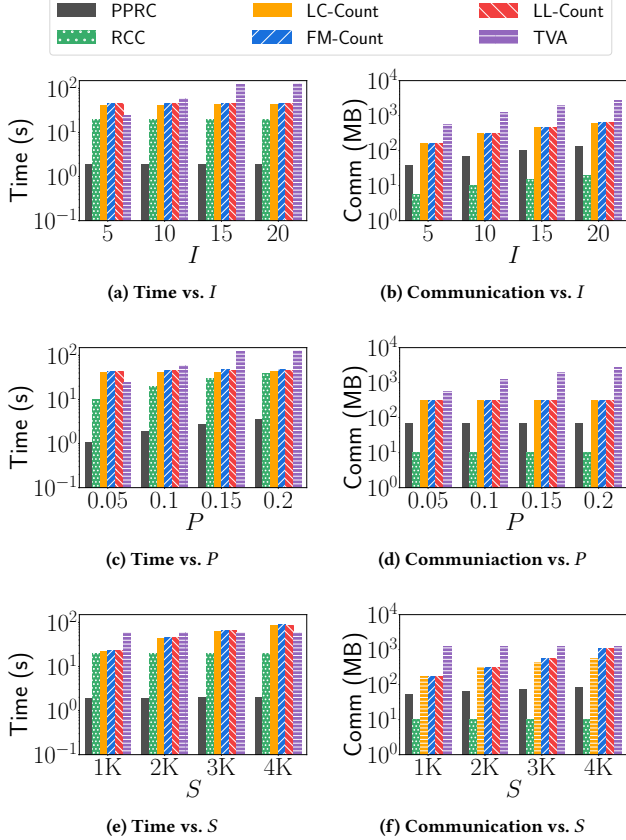


Figure 13: Efficiency comparison with various I , P , and S .

setting in [42], we use $k_0 = 4096$, $k_1 = 80$, and $k_2 = 160$, yielding an 8192-bit public modulus and approximately 1 KB per ciphertext. Therefore, the total encrypted OLC sketch size grows linearly with the sketch length S . For the sketch sizes evaluated in our experiments ($S \in \{1K, 2K, 3K, 4K\}$), the corresponding encrypted sketch sizes are approximately 1 MB, 2 MB, 3 MB, and 4 MB, respectively.

Primitive Operation Cost Analysis. To further characterize the computational behavior of the adopted SHE implementation [26], we measure the latency of its core homomorphic operations. Table 5 reports the average execution time of each primitive over 1,000 independent trials.

The results reveal two important observations. First, ciphertext-plaintext operations (Add-II and Mul-II) consistently outperform ciphertext-ciphertext operations. Second, ciphertext-plaintext multiplication (Mul-II) reduces latency by more than 96% compared

with ciphertext-ciphertext multiplication (Mul-I). This validates our design choice of minimizing expensive ciphertext-ciphertext multiplications in PPRC whenever possible.

Table 5: Computational Cost of Cryptographic Primitive Operations.

Operation	Notation	Latency ($\mu\text{s/op}$)
Add-I	$E(m_1) + E(m_2) \rightarrow E(m_1 + m_2)$	0.161
Add-II	$E(m_1) + m_2 \rightarrow E(m_1 + m_2)$	0.120
Mul-I	$E(m_1) \cdot E(m_2) \rightarrow E(m_1 \cdot m_2)$	6.719
Mul-II	$E(m_1) \cdot m_2 \rightarrow E(m_1 \cdot m_2)$	0.231

Detailed Cost Dissection. To better understand the runtime bottlenecks, we analyze the end-to-end latency of PPRC by separating it into five components: query generation, query transmission, PRP evaluation, OLC aggregation, and decryption (Fig. 14).

As shown in Fig. 14(a), increasing the number of participants I from 5 to 20 raises the OLC aggregation cost only slightly, from 0.019 s to 0.059 s, indicating that homomorphic aggregation introduces minimal centralized overhead. The other components remain nearly unchanged, confirming that system scalability is not limited by the aggregation stage.

In Fig. 14(b), increasing the sampling proportion P from 0.05 to 0.2 mainly affects PRP evaluation, whose runtime grows linearly from 0.51 s to 2.02 s due to the larger number of local point-in-range checks. Meanwhile, the costs of query generation, network transmission, OLC aggregation, and decryption remain nearly constant, contributing less than 0.25 s in total.

A similar pattern is observed in Fig. 14(c). Increasing the query range primarily affects client-side query generation, increasing its cost from 0.068 s to 0.335 s, while PRP evaluation remains stable at approximately 2.01 s. These results show that the dominant cost of PPRC lies in localized PRP evaluation, while the remaining components scale efficiently with both system size and query range.

D IMPLEMENTATION DETAILS AND BFV INSTANTIATION OF PPRC

In this section, we provide the concrete system implementation details of the PPRC protocol instantiated under the Fan-Vercauteren (BFV) fully homomorphic encryption scheme. The entire system is engineered in native C++17, utilizing Microsoft SEAL (v4.1) for homomorphic primitives.

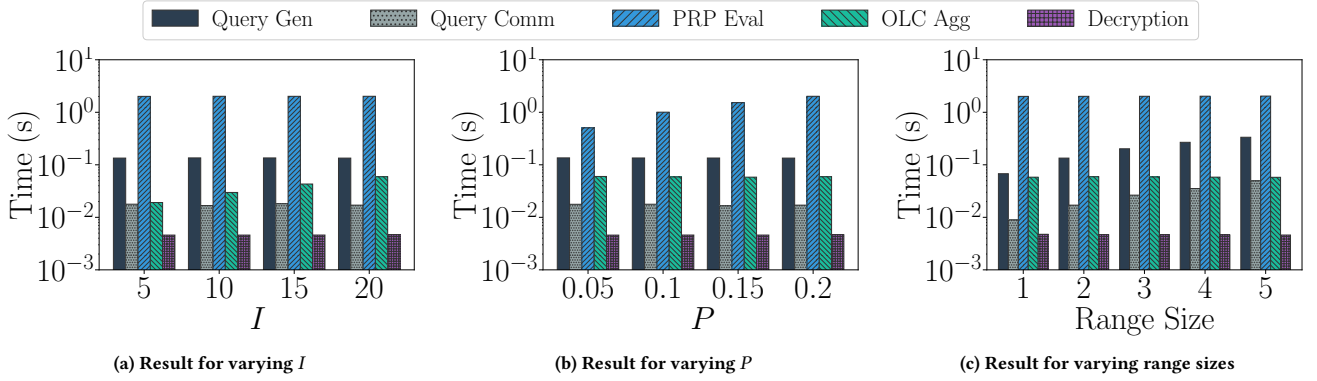


Figure 14: Detailed runtime analysis of PPRC under varying system parameters.

D.1 Cryptographic Execution Framework and Parameterization

In our BFV-based instantiation, the secret key sk is strictly localized within the Query User’s (QU) secure domain, while the public evaluation keys—including the public key pk , relinearization keys rk , and Galois keys gk —are distributed to the Central Aggregator (CA) and Data Holders (DHs).

Cryptographic Parameterization: The foundational security and performance of the BFV scheme rely strictly on the selection of the polynomial modulus degree N and the structural modulus primes. In our implementation, we parameterize the BFV context with $N = 8192$ and establish the coefficient modulus using the default secure prime shield recommended by the homomorphic encryption standard (`seal::CoeffModulus::BFVDefault`), which guarantees at least 128 bits of security against known lattice-based attacks. A critical engineering challenge in evaluating the Private Range Predicate (PRP) is preserving a sufficient invariant noise budget while executing sequential homomorphic multiplications. To achieve this, we precisely constrain the plaintext space by selecting a 17-bit batching prime (`seal::PlainModulus::Batching(8192, 17)`). This specific parameterization bounds the plaintext modulus closely to 49153, operating at the mathematical limit for $N = 8192$. This configuration reserves maximum capacity for the noise budget, ensuring that ciphertext-ciphertext multiplications during predicate evaluations do not trigger decryption failures.

Range Encoding: To encode the spatial query range dynamically without leaking dimensional lengths to the DHs, the QU constructs dimensional Bloom filters. The optimal bit-array length M is mathematically derived at runtime based on the target false-positive rate f_p and the expected spatial element count μ . With a fixed hash function count $K = 3$, the length is dynamically allocated as $M = \lceil -K\mu / \ln(1 - f_p^{1/K}) \rceil$.

Asynchronous Network Framing: The architectural data flow between the QU, the CA, and the DHs requires transmitting massive serialized ciphertext arrays. To prevent TCP stream fragmentation and ensure payload integrity, our network layer is powered by `boost::asio`. We implement a strictly length-prefixed framing protocol, where every serialized payload is prepended with a 64-bit unsigned integer (`uint64_t`) denoting its exact byte length. This

guarantees that the receiving nodes can dynamically allocate accurate memory buffers prior to block ingestion, circumventing memory segmentation faults and enabling highly efficient zero-copy ciphertext parsing during the homomorphic aggregation phase.

D.2 SIMD-Accelerated Range Evaluation and Aggregation

The Private Range Predicate (PRP) phase mitigates the overhead of individual bit encryption by packing the dimensional Bloom filters (BF_x and BF_y) directly into the SIMD slots of BFV ciphertexts utilizing `seal::BatchEncoder`. Upon receiving these packed ciphertexts, each DH i must evaluate the point-in-range conditions for its local records entirely in the encrypted domain. Since individual slots within a packed ciphertext cannot be accessed or extracted directly, the DH leverages Galois keys (gk) to perform homomorphic cyclic slot rotations (`rotate_rows`). Specifically, to simultaneously evaluate the K hash positions for a batch of local records, the DH cyclically shifts the packed Bloom filter ciphertext to align the target encrypted bits with the active evaluation slots. Following this alignment, the DH executes a sequence of homomorphic ciphertext-ciphertext multiplications across the rotated states. This procedure homomorphically computes the logical AND operation over the K hashed indices, yielding a unified encrypted in-range indicator $E(L_n^{(i)})$ for each record $o_n^{(i)}$, where $L_n^{(i)} = 1$ indicates a valid spatial match and 0 otherwise.

During the subsequent Oblivious Linear Counting (OLC) phase, each DH i maps its records into an encrypted sketching vector $E(LC_i)$ of length S via homomorphic additions governed by the computed indicators:

$$E(LC_i[h(o_n^{(i)})]) = E(LC_i[h(o_n^{(i)})]) + E(L_n^{(i)}) \quad (6)$$

The CA then homomorphically aggregates these local sketches across all I DH sites by computing $E(LC[s]) = \sum_{i=1}^I E(LC_i[s])$. To guarantee data privacy against a curious but compliant (semi-honest) CA, a dual-layer perturbation suite is enforced: counter-index obfuscation (structural index permutation) and non-zero blinding (homomorphic plaintext multiplication by a random integer r_s). Finally, the QU decrypts the perturbed aggregated sketch, identifies the total number of unoccupied zero slots (S'), and derives the precise aggregate range count utilizing the standard Linear Counting estimator $\hat{RC} = -S \ln(S'/S)$.

Algorithm 4: Aggregation Phase of LC-Count

Input : Dataset $\{(x_n^{(i)}, y_n^{(i)}, E(L_n^{(i)}))\}_{n=1}^{N_i}$ held by each data holder $i \in [1, I]$.
Output : Range count RC obtained by the query user.
/* - Local Sketch Construction by data holders - */
1 Each data holder i initializes each counter of its LC sketch
 $LC_i[s] = 0$ for all $s \in [1, S]$ and encrypts each counter into
 $E(LC_i[s])$ using pk ;
2 All data holders jointly agree on a hash function h ;
3 **for** DH $i = 1$ **to** I **do**
4 **for** $n = 1$ **to** N_i **do**
5 $s = h(x_n^{(i)}, y_n^{(i)})$;
6 $E(LC_i[s]) = E(LC_i[s]) + E(L_n^{(i)})$;
7 **for** $s = 1$ **to** S **do**
8 $E(LC_i[s]) = \text{ZeroTest}(E(LC_i[s]))$; // ZeroTest:
 maps $E(0) \rightarrow E(0)$, others $\rightarrow E(1)$
/* - Secure Aggregation by the Central
 Aggregator - */
9 Each data holder i sends $E(LC_i)$ to the central aggregator;
10 **for** $s = 1$ **to** S **do**
11 $E(LC[s]) = \sum_{i=1}^I E(LC_i[s])$;
12 $E(LC[s]) = \text{ZeroTest}(E(LC[s]))$;
13 Compute $E(S') = S - \sum_{s=1}^S E(LC[s])$;
/* - Decryption and Estimation - */
14 The central aggregator sends $E(S')$ to the query user;
15 The query user decrypts $E(S')$ into S' using sk ;
16 The query user computes $RC = -S \ln(\frac{S'}{S})$;

E LC-COUNT

In this section, we describe the LC-Count protocol. The Setup and Range Evaluation phases are identical to those in PPRC; thus, we focus on the Aggregation phase, whose pseudocode is shown in Algorithm 4. This phase consists of three steps:

- **Step 1 (Lines 1-8): Sketch Construction.** Each data holder i initializes its LC sketch LC_i with S encrypted counters, each set to $E(0)$ using the public key pk . All data holders jointly agree on a hash function h that uniformly maps each record $(x_n^{(i)}, y_n^{(i)})$ to an index in $\{1, \dots, S\}$. For each record, the data holder adds its encrypted range indicator $E(L_n^{(i)})$ to the corresponding counter as $E(LC_i[h(x_n^{(i)}, y_n^{(i)})]) = E(LC_i[h(x_n^{(i)}, y_n^{(i)})]) + E(L_n^{(i)})$. After all updates, $E(LC_i)$ is normalized using the $\text{ZeroTest}(\cdot)$ operation [43], which is based on Lagrange interpolation and transforms $E(0) \rightarrow E(0)$ while mapping all non-zero ciphertexts to $E(1)$. This normalization ensures that each counter reflects whether any record was hashed into its position.
- **Step 2 (Lines 9-13): Secure Aggregation.** Each data holder sends its encrypted sketch $E(LC_i)$ to the central aggregator. The aggregator performs element-wise homomorphic addition to aggregate all sketches: $E(LC[s]) = \sum_{i=1}^I E(LC_i[s])$, $s \in [1, S]$. It then applies $\text{ZeroTest}(\cdot)$ again to normalize all counters and

computes the encrypted number of zero counters as $E(S') = S - \sum_{s=1}^S E(LC[s])$.

- **Step 3 (Lines 14-16): Decryption and Estimation.** The central aggregator sends $E(S')$ to the query user, who decrypts it to obtain S' . The query user then estimates the final range count using the standard LC estimator $RC = -S \ln(S'/S)$.

Algorithm 5: Aggregation Phase of FM-Count

Input : Dataset $\{(x_n^{(i)}, y_n^{(i)}, E(L_n^{(i)}))\}_{n=1}^{N_i}$ held by each data holder $i \in [1, I]$.
Output : Range count RC obtained by the query user.
/* - Local Sketch Construction by data holders - */
1 Each data holder i initializes each counter of its FM sketch
 $FM_i[c][w] = 0$ for all $c \in [1, C]$, $w \in [1, W]$ and encrypts
 each counter into $E(FM_i[c][w])$ using pk ;
2 All data holders jointly agree on C independent hash
 functions $\{h_1, \dots, h_C\}$;
3 **for** DH $i = 1$ **to** I **do**
4 **for** $n = 1$ **to** N_i **do**
5 **for** $c = 1$ **to** C **do**
6 $w_c = h_c(x_n^{(i)}, y_n^{(i)})$;
7 $E(FM_i[c][w_c]) = E(FM_i[c][w_c]) + E(L_n^{(i)})$;
8 **for** $c = 1$ **to** C **do**
9 **for** $w = 1$ **to** W **do**
10 $E(FM_i[c][w]) = \text{ZeroTest}(E(FM_i[c][w]))$;
 // ZeroTest: maps $E(0) \rightarrow E(0)$, others
 $\rightarrow E(1)$
/* - Secure Aggregation by the Central
 Aggregator - */
11 Each data holder i sends $E(FM_i)$ to the central aggregator;
12 **for** $c = 1$ **to** C **do**
13 **for** $w = 1$ **to** W **do**
14 $E(FM[c][w]) = \sum_{i=1}^I E(FM_i[c][w])$;
15 $E(FM[c][w]) = \text{ZeroTest}(E(FM[c][w]))$;
16 **for** $c = 1$ **to** C **do**
17 **for** $w = 2$ **to** W **do**
18 $E(FM[c][w]) = E(FM[c][w]) \cdot E(FM[c][w-1])$;
19 Compute $E(Z_N) = \sum_{c=1}^C \sum_{w=1}^W E(FM[c][w])$;
/* - Decryption and Estimation - */
20 The central aggregator sends $E(Z_N)$ to the query user;
21 The query user decrypts $E(Z_N)$ into Z_N using sk ;
22 The query user computes $RC = 2^{\frac{Z_N}{C}} / \phi$, where $\phi \approx 0.77351$;

F FM-COUNT

In this section, we present the FM-Count protocol. Similar to LC-Count, its Setup and Range Evaluation phases remain unchanged from PPRC; hence, we focus on describing the Aggregation phase, whose pseudocode is provided in Algorithm 5. This phase consists of three steps:

Algorithm 6: Aggregation Phase of LL-Count

Input : Dataset $\{(x_n^{(i)}, y_n^{(i)}, E(L_n^{(i)}))\}_{n=1}^{N_i}$ held by each data holder $i \in [1, I]$.
Output: Range count RC obtained by the query user.
/* – Local Sketch Construction by data holders – */

- 1 Each data holder i initializes each counter of its LL sketch $LL_i[c][w] = 0$ for all $c \in [1, C]$, $w \in [1, W]$ and encrypts each counter using pk ;
- 2 All data holders jointly agree on $C + 1$ independent hash functions $\{h, h_1, \dots, h_C\}$;
- 3 **for** DH $i = 1$ **to** I **do**
- 4 **for** $n = 1$ **to** N_i **do**
- 5 $c = h(x_n^{(i)}, y_n^{(i)})$;
- 6 $w = h_c(x_n^{(i)}, y_n^{(i)})$;
- 7 $E(LL_i[c][w]) = E(LL_i[c][w]) + E(L_n^{(i)})$;
- 8 **for** $c = 1$ **to** C **do**
- 9 **for** $w = 1$ **to** W **do**
- 10 $E(FM_i[c][w]) = \text{ZeroTest}(E(FM_i[c][w]))$;
 // ZeroTest: maps $E(0) \rightarrow E(0)$, others
 $\rightarrow E(1)$
- /* – Secure Aggregation by the Central Aggregator – */
- 11 Each data holder sends $E(LL_i)$ to the central aggregator;
- 12 **for** $c = 1$ **to** C **do**
- 13 **for** $w = 1$ **to** W **do**
- 14 $E(LL[c][w]) = \sum_{i=1}^I E(LL_i[c][w])$;
- 15 $E(LL[c][w]) = 1 - \text{ZeroTest}(E(LL[c][w]))$;
- 16 **for** $c = 1$ **to** C **do**
- 17 **for** $w = W-1$ **to** 1 **do**
- 18 $E(LL[c][w]) = E(LL[c][w]) \cdot E(LL[c][w+1])$;
- 19 Compute $E(Z_N) = C(W-1) - \sum_{c=1}^C \sum_{w=1}^W E(LL[c][w])$;
- /* – Decryption and Estimation – */
- 20 The central aggregator sends $E(Z_N)$ to the query user;
- 21 The query user decrypts $E(Z_N)$ into Z_N using sk ;
- 22 The query user computes $RC = \alpha_C \cdot C \cdot 2^{Z_N/C}$, where
$$\alpha_C = \left(\Gamma(-\frac{1}{C}) \cdot \frac{1-2^{1/C}}{\log 2}\right)^C$$
;

- **Step 1 (Lines 1-10): Sketch Construction.** Each data holder i constructs an encrypted FM sketch $E(FM_i)$ composed of C arrays, and each array contains W encrypted counters. All counters are initialized to $E(0)$ using the public key pk . All data holders jointly agree on C independent hash functions $\{h_1, \dots, h_C\}$. For each record $(x_n^{(i)}, y_n^{(i)}, E(L_n^{(i)}))$, data holder i computes C hash indices $w_c = h_c(x_n^{(i)}, y_n^{(i)})$ for $c = 1, \dots, C$, and updates the sketch as $E(FM_i[c][w_c]) = E(FM_i[c][w_c]) + E(L_n^{(i)})$ for

$c = 1, \dots, C$. After processing all records, $\text{ZeroTest}(\cdot)$ is applied to normalize each counter such that $E(FM_i[c][w]) = E(1)$ if any record contributes to the position, and $E(0)$ otherwise.

- **Step 2 (Lines 11-19): Secure Aggregation.** Each data holder i sends $E(FM_i)$ to the central aggregator, which aggregates them counter-wise as $E(FM[c][w]) = \sum_{i=1}^I E(FM_i[c][w])$ for $c = 1, \dots, C$ and $w = 1, \dots, W$. The aggregator applies $\text{ZeroTest}(\cdot)$ again to ensure that all aggregated counters remain binary. It then performs a prefix-wise cumulative multiplication over each row to propagate leading zeros, as in the original FM sketch design. Finally, it computes $E(Z_N) = \sum_{c=1}^C \sum_{w=1}^W E(FM[c][w])$.
- **Step 3 (Lines 20-22): Decryption and Estimation.** The central aggregator sends $E(Z_N)$ to the query user, who decrypts it to obtain Z_N . The final range count is estimated using the standard FM sketch estimator $RC = 2^{Z_N/C} / \phi$, where $\phi \approx 0.77351$ is a bias-correction constant.

G LL-COUNT

In this section, we present the LL-Count protocol. Similar to LC-Count, its Setup and Range Evaluation phases remain unchanged from PPRC, and we thus focus on the Aggregation phase. The pseudocode is shown in Algorithm 6, consisting of three main steps:

- **Step 1 (Lines 1-10): Sketch Construction.** Each data holder i constructs an encrypted LL sketch $E(LL_i)$ composed of C arrays, and each array contains W encrypted counters. All counters are initialized to $E(0)$ using the public key pk . The data holders jointly agree on $C+1$ independent hash functions $\{h, h_1, \dots, h_C\}$, where h selects the target array and h_c determines the position within that array. For each $(x_n^{(i)}, y_n^{(i)}, E(L_n^{(i)}))$, data holder i computes $c = h(x_n^{(i)}, y_n^{(i)})$ and $w = h_c(x_n^{(i)}, y_n^{(i)})$, and updates the encrypted counter as $E(LL_i[c][w]) = E(LL_i[c][w]) + E(L_n^{(i)})$. Finally, $\text{ZeroTest}(\cdot)$ is applied to normalize each counter such that $E(LL_i[c][w]) = E(1)$ if any record contributes to the position, and $E(0)$ otherwise.
- **Step 2 (Lines 11-19): Secure Aggregation.** Each data holder sends $E(LL_i)$ to the central aggregator, which aggregates them counter-wise: $E(LL[c][w]) = \sum_{i=1}^I E(LL_i[c][w])$. The central aggregator then converts the counters via $E(LL[c][w]) = 1 - \text{ZeroTest}(E(LL[c][w]))$, ensuring that each position reflects whether at least one record across all data holders was mapped there. Next, a backward cumulative multiplication is applied along each row to propagate the rightmost one bit toward the left, consistent with the LL sketch procedure. Finally, the central aggregator computes $E(Z_N) = C(W-1) - \sum_{c=1}^C \sum_{w=1}^W E(LL[c][w])$.
- **Step 3 (Lines 20-22): Decryption and Estimation.** The central aggregator sends $E(Z_N)$ to the query user, who decrypts it using the secret key sk to obtain Z_N . The query user then estimates the range count using the standard LL sketch estimator $RC = \alpha_C \cdot C \cdot 2^{Z_N/C}$, where $\alpha_C = \left(\Gamma(-\frac{1}{C}) \cdot \frac{1-2^{1/C}}{\log 2}\right)^C$ is the standard bias-correction factor.